

Feeling the Spell: Enhancing Embodiment in VR via Haptic gloves

Dushan Oravskyi

Cedric Greiten

Yurii Pashkevych

Maximilian Adolenko

Technische Universität Dortmund

Dortmund, Germany

dushan.oravskyi@tu-dortmund.de

cedric.greiten@tu-dortmund.de

yurii.pashkevych@tu-dortmund.de

maximilian.adolenko@tu-dortmund.de

Abstract

Inspired by prior research regarding the effect of VR gloves on user embodiment [9], we implemented a cost-effective haptic VR glove based on the LucidGloves design [5]. To evaluate the impact of the gloves on embodiment, we developed an interactive, fantasy-themed virtual reality demonstration in Unity. Additionally, to further test the multimodal aspect of embodiment, we integrated an existing speech recognition system into the demonstration.

CCS Concepts

• **Human-centered computing** → **Human computer interaction (HCI)**.

Keywords

human computer interaction

1 Introduction

Embodiment represents the state in which a user accepts a virtual body part, such as a hand, as their own [9]. Recent studies investigate how the degree of natural mapping between physical input devices and their virtual counterparts influences this sense of embodiment [9]. These studies conclude that VR gloves yield highly promising results, as they offer the highest level of natural mapping among tested devices by directly translating physical hand movements into the virtual environment.

One of the limitations of those studies, however, was the lack of haptic feedback in the tested Noitom Hi5 VR gloves. Another limitation was the cost of these VR gloves, which can exceed one thousand dollars [1]. This motivated us to build haptic VR gloves using cost-effective materials, based on the open-source design [5].

To evaluate embodiment with these gloves, we developed an interactive demonstration scene in Unity. Our goal was not only to test the hardware's haptic capabilities but also to create a fitting atmosphere that maximizes immersion. Initially, we planned to theme the environment around tabletop role-playing games, similar to the approaches in [7] and [8]. However, after careful consideration, we pivoted to a broader fantasy setting, as the social dynamics during

tabletop gameplay might not translate well into the virtual environment. More specifically, we designed a spell-casting scenario where the user can not only physically feel the components but also actively create and cast spells with them.

Furthermore, we wanted to explore how multimodality influences immersion. To achieve this, we integrated a speech recognition system that reacts to the user's spoken commands and triggers the corresponding spell.

2 Implementation Details

2.1 Glove

To implement the physical glove, the "LucidGloves" [3] design was replicated. These gloves enable finger tracking and haptic force feedback.

2.1.1 How it works. To enable finger tracking and force feedback, different components are used. Potentiometers, which consist of a rotary knob, are used for finger tracking. Depending on the position of the rotary knob, the potentiometer outputs a voltage between 0 and a maximum value. The potentiometers are spring-loaded, so they always return to a default position, which represents an uncurled finger when the knob is not rotated. The spring-loaded potentiometer is then connected to a finger via a wire. If a finger is curled, the knob is rotated and the curl of the finger can be measured through the output voltage of the potentiometer. For each finger, one spring-loaded potentiometer is provided. Figure 1 shows the assembled potentiometer module from the inside. Figure 2 shows the potentiometer connected to a wire. To enable force feedback, servo motors are attached to the potentiometer module. In addition, a mechanical stopper, shown in Figure 3, is attached to the rotary knob. By adding these components to the potentiometer module, force feedback can be enabled by rotating the servo motor to the position needed to simulate the feeling of touching an object. The rotation of a servo motor then blocks the mechanical stopper, so a finger cannot be curled anymore and force feedback is simulated in this way. For controlling all components of the glove and communicating with the host computer, which is running the VR environment, a microcontroller is used. To use the gloves in the VR environment, the "OpenGloves" driver [4] is provided.



This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.

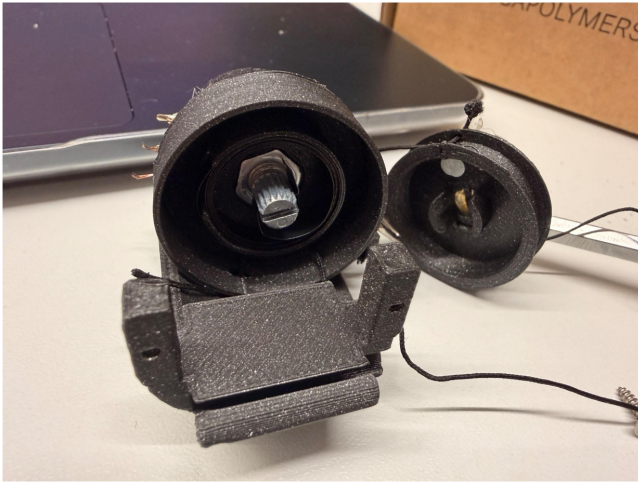


Figure 1: Potentiometer module from the inside

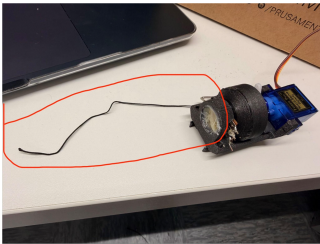


Figure 2: Potentiometer module connected to a wire

2.1.2 How it was built. To build the glove, all needed parts were ordered and 3D-printed. The glove was assembled by soldering everything together. Figure 4 shows the assembled glove.

2.1.3 Problems during building. Following the instructions of "LucidGloves" [3], there were several problems that had to be fixed. The first problem that occurred was that the 3D-printed parts are usually designed to simply click together. But with our own 3D-printed parts this did not work. The exact reason could not be determined. Possible reasons could be that the print quality itself was not sufficient or that the provided STL files were of poor quality. This problem was solved by welding or gluing the parts together. Another problem was the firmware running on the microcontroller of the glove. After initially assembling it, the finger tracking was not working properly. The first reason for this inconvenience was that the wiring in the wiring diagram, which was provided, was different from the wiring used in the firmware. To solve this problem, the firmware's wiring was adjusted to match the wiring in the wiring diagram. After adjusting the wiring, still no finger curl was measured. The firmware always measured that there was no finger curl at all. The first thought was that the potentiometers were broken. To test them, a small script that read and printed the output voltage of each potentiometer was uploaded to the microcontroller. The result was that the potentiometers worked. Therefore, the firmware was investigated further. The firmware worked like this: First it reads the potentiometer outputs for each finger. It also keeps track



Figure 3: Servo motor attached to the module

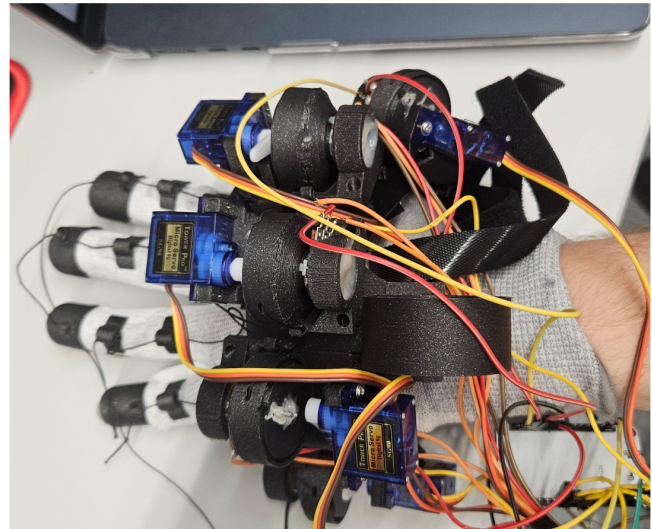


Figure 4: Assembled glove

of the maximum and minimum value that can be reached, so the firmware knows when a finger is fully curled or not curled. After reading the output, the firmware normalizes the read output to a value between 0 (no finger curl) and 4095 (max finger curl) using the tracked minimum and maximum. The reason for this is that the measured value has no meaning for the driver in its raw form. The reason for this is that for example it differs for each finger although they have the same curl because the value depends for example on the distance a finger can curl. After normalizing, the driver knows how much the finger is curled and this curl can be used in the VR environment. The firmware also stored the tracked range between maximum and minimum value of each finger in the EEPROM, a persistent memory, so there would be no need for the glove to calibrate after one successful calibration. To remember that some values were stored, a flag was also stored. After flashing the firmware, the flag indicated that there were stored values although there was no calibration before. Because of this, the firmware did not calibrate properly and could not measure any finger curl properly. After deleting the use of the EEPROM, curl measurement worked correctly. Another problem occurred during testing and connecting the glove to Unity. The problem was that the design of the glove was not sufficiently stable. Because of this, the glove had to be reassembled several times; especially cables had to be resoldered often.

2.2 Audio

Besides the physical feedback used for the feeling of embodiment, we wanted to add another interactive component to our system, so that the player could feel even more immersed. Naturally, we settled on speech recognition that detects what a player is saying and translates it into a command. For a spell to be cast, the right word(s) have to be spoken.

2.2.1 Analyzing the audio. To turn speech into text, we began writing a Python script using the *Vosk* model. Vosk was chosen over alternatives like Whisper, as we did not need top of the line speech recognition for our purposes, as commands would only consist of a few words maximum, and could save on resources at the same time. This would improve performance of the demo on older hardware and therefore player experience.

The script reads audio input from a specified input device using python’s *sounddevice* library. To find the right device, one would execute the script with the `-list-devices` flag. This would result in a list of installed audio devices with their IDs being outputted and the execution being prematurely stopped. This feature was later made available through the GUI in Unity. By default, the model expects speech coming in the English language (US). This can be changed through the use of `-language` argument as seen in figure 5.

We integrated Vosk into the script and noticed that there was always a noticeable delay of about 1 to 2 seconds before the spoken command was processed. As this could potentially lead to a disconnect between the player and the game, we investigated this problem. It turned out, that when using Vosk’s default *Result*, the behavior was to wait to see if the user would continue speaking, presumably so that no sentence is ever split in the middle. As it was likely we would not need this strict behavior, we switched the code

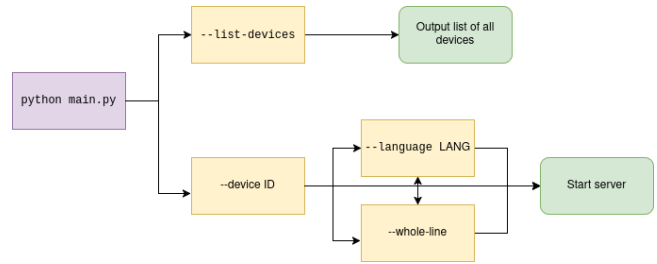


Figure 5: Diagram of the command structure of the python script

to *PartialResult*. This data type returns much quicker, at the cost of sometimes containing wrong words at first, before correcting itself. This means that sometimes wrong words get sent through the later constructed pipeline to the demo, but this was an acceptable trade-off for our use case, as the risk for overlap between commands and misheard words was likely very small.

2.3 Unity

2.3.1 Demo. To provide a meaningful demonstration of the glove, a demo application was created in Unity. The demo is set in a fantasy-inspired scene in which the user must pick up spell components, mix them together to create spells, and then cast those spells.

2.3.2 Initial Setup. Work in Unity began with the initial project setup. First, a new Unity project was created and the Unity version control system “Plastic SCM” was configured. Afterwards, the *XR Interaction Toolkit* and *XR Hands* packages were installed, and all relevant project settings were adjusted to enable VR support in Unity. Finally, the prebuilt XR Origin with hand tracking was added to the scene. The XR Origin is a prefab that contains the camera, controllers, hands, and other components required for VR.

2.3.3 Main Features. The main features of the demo are:

- an *inventory* from which the spell components can be taken,
- a so-called *spell surface* (later referred to as the *alchemy circle*), which is used to mix the components together, and
- a *spell casting system*, which allows the user to cast spells.

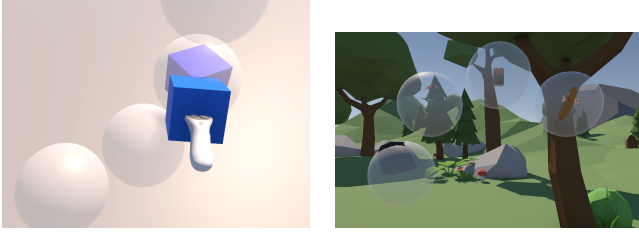
2.3.4 Inventory. As a basis for the inventory, we used the *XR Socket Interactor* component, which allows the user to store interactable objects in a specific area. The inventory itself is a circle containing a number of these sockets (hereafter referred to as *slots*), in which items can be stored.

The inventory has effectively infinite capacity. When it is empty and the user opens it for the first time, it initially provides only two slots. As soon as the rightmost slot is filled, a new slot is added along the circle. Once the circle reaches its maximum number of slots, a new outer circle appears, and this process continues.

The inventory can be opened and closed using special gestures, which are discussed later in Section 2.3.8. When opening or closing the inventory, an animation is shown in which the inventory appears from or disappears into the user’s wrist.

The inventory also supports a *creative mode*, in which the user immediately sees all items available in the demo and can take as

many items as desired. This mode is used as the normal mode during the demo.



(a) Putting an item into the inventory (b) Inventory with items in creative mode

Figure 6: Inventory interaction and creative mode

2.3.5 Spell Creation. Initially, the spell creation system was implemented as a so-called *spell surface*: a floating table in the air where the user could place components. As soon as the required components for a spell were present, the spell was created and the user could pick it up. When the spell was taken, the components were consumed.

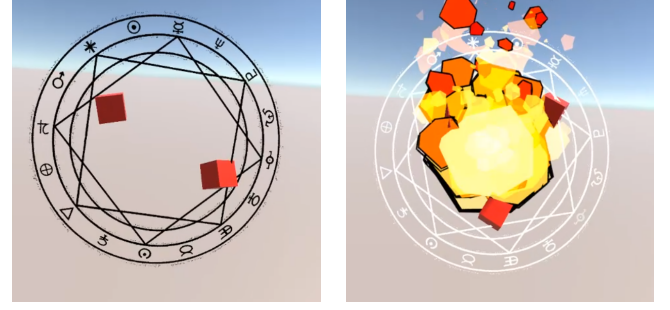
Subsequently, the system was improved to make it more immersive. First, the spell surface was replaced by a so-called *alchemy circle*, which is a circle that can be conjured in mid-air using a gesture (discussed later in Section 2.3.9). Its animation is as follows: upon performing the gesture, the circle appears at the location where the gesture was executed, with an animation that fills the circle in the clockwise direction. In its idle state, the circle rotates slowly and its lines are black.

The user can place items in the area near the circle or throw them into the circle. As soon as an item enters the vicinity of the circle, it smoothly flies to the correct position on the circle. When more than one item is added to the circle, all items smoothly rearrange themselves to maintain a symmetric arrangement. Items can also be removed from the circle by taking them out; in this case, the remaining items again smoothly fly to their correct positions to preserve symmetry.

When enough items for a spell are present and the name of the spell is spoken, the circle rotates faster and turns white, and the spell is created. As soon as the spell is taken, the items are consumed.

2.3.6 Spell Casting. Initially, spells were implemented as a base class containing all functions we anticipated a spell might need. Each concrete spell had to inherit from this base class and implement its specific functionality. Over time, however, we realized that this approach was not very flexible and led to significant code duplication. Consequently, we decided to implement a more flexible system.

The architecture of the new system is portrayed in Figure 8: there is a base *SpellObject* class that connects two components that every spell must have: *SpellBehavior* and *Hit*. The *SpellBehavior* component describes how the spell behaves. For example, the *Throwable* behavior describes all spells that are thrown; most of our spells, such as *Fireball* or *Lightning Spear*, can already be described by this



(a) Deactivated alchemy circle (b) Activated alchemy circle

Figure 7: Alchemy circle in deactivated and activated states

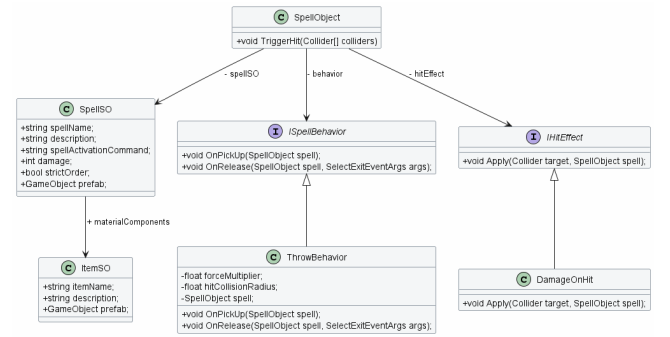


Figure 8: Spell architecture

behavior. The *Hit* component describes what happens when the spell registers a hit, e.g., *DamageOnHit* or *SpawnOnHit*.

Each spell can have exactly one behavior but multiple hit components. For instance, the *Fireball* spell uses the *Throwable* behavior together with *DamageOnHit* and *SpawnOnHit* hit components. When it hits something, it deals damage and spawns a fire effect. The main advantage of this system is its flexibility: many new spell variants can now be created without writing additional code, directly within Unity.

2.3.7 Supporting Features. To support the main features and to make the demo experience more immersive and engaging, we implemented several additional features and systems. These include multiple gesture-based systems—directional movement recognition, gesture combos, and complex movement recognition—as well as a grasping feature, a user interface, a dummy for spell testing, and other features.

2.3.8 Gesture Combo. The built-in XRI Toolkit *Static Hand Gesture* component can only detect a single static gesture, but we wanted to detect sequences of gestures. We used the solution from the *SimpleGestureRecorder* repository [10]. In addition to the main gesture recording feature, it provides a gesture combo system, which we adopted and improved.

As a basis, each gesture uses code that is almost identical to the *Static Hand Gesture* component. However, it is also connected to another component that allows you to save a list of gestures and,

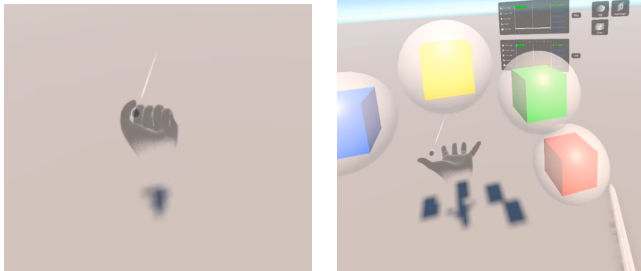
after performing one gesture, waits for the next. If all gestures are performed in the correct order, an event is triggered.

We improved the system to better fit our scenario by changing when the timer for the next gesture starts. Originally, the timer started immediately after the gesture was performed; we changed it to start only after the gesture was released. This allows the user to hold the first gesture for as long as desired and then perform the next one.

In the demo, this system is used for opening and closing the inventory:

- When the user points a fist upwards and then opens the hand, the inventory appears.
- When the user points an open palm upwards and then closes the hand, the inventory disappears.

Additionally, if the open gesture is performed while the inventory is already open, the inventory is simply moved to the current hand position.



(a) Fist pointed upwards to start/complete the gesture (b) Open palm pointed upwards to complete/start the gesture

Figure 9: Gesture sequence for opening/closing the inventory

2.3.9 Complex Movement Recognition. We also wanted to add functionality to recognize complex movements drawn in the air, effectively allowing the user to draw sigils in mid-air. For this purpose, we implemented a system based on the *\$1 Point-Cloud Gesture Recognizer* asset [11]. This algorithm can recognize prerecorded symbols in 2D. The obvious problem is that our application operates in 3D.

To solve this problem, we implemented a system that projects the 3D movement onto the screen and then uses the 2D algorithm to recognize the symbol. The user draws a symbol in the air by making points with a finger gesture. As soon as the gesture is released, the system recognizes the symbol with a certain confidence value and triggers an event if the confidence is sufficiently high.

We also added visual feedback for the actual “drawing”: as soon as the user starts drawing, a line follows their finger. When the gesture is released, the line smoothly shrinks to the last point and disappears.

2.3.10 Directional Movement Recognition. We also wanted to recognize directional movements, such as hand swipes, so we implemented an additional system for directional movement recognition. The system works as follows: when the user starts moving their hand in a specific direction, the system begins tracking the movement direction. As soon as the user has moved far enough in that



Figure 10: Drawing a circle to spawn an alchemy circle

direction, an event is triggered. This mechanism also supports rotations and can be combined with other gestures.

In the demo, this system is used to make the alchemy circle disappear: the user simply swipes through the circle from right to left, as if erasing it with their hand.

2.3.11 Dummy. To enable testing of spell effects, we implemented a dummy that can take damage. It also features a health bar that displays the dummy’s current health and is always oriented towards the user.

2.3.12 Grasping. In the standard XRI Toolkit, there is no natural grasping interactor. An interactable object can only be either grabbed or not, and there is no built-in way to trigger this using natural hand movements; only specific pre-baked gestures are supported. For example, the default XR Hand Origin uses a pinch gesture for grabbing, and the object simply snaps into the hand.

However, we needed a form of natural grasping, or alternatively a pre-baked gesture and hand position for each object, in order to make the force feedback work as intended. For this reason, we implemented our own system on top of the *XR Direct Interactor*.

Initially, we attempted to implement a scoring system: if the score was high enough, the interactable would be grabbed; if it was low enough, it would be released. This approach failed quickly. The main problems were that some properties are obligatory while others are optional, and it was difficult to define how much score each property should contribute.

Consequently, we switched to a heuristic method. The core idea is to gather information about the current state of the hand and then decide whether this state is sufficient to consider the item grabbed. We defined the following criteria:

- whether the thumb is touching the object,

- whether the thumb is curled sufficiently,
- which fingers are touching the object and how many of them,
- which fingers are curled sufficiently and how many of them,
- whether the palm is near enough to the object.

Based on these criteria, we defined two states in which an object can be grabbed: *pinch grab* and *power grab*. A pinch grab occurs when the user picks up an object using the thumb and fingers, whereas a power grab occurs when the user uses the palm instead of the thumb.

2.3.13 Spells. For the demo, four spells were implemented: Fireball, Green Fireball, Fire Arrow, and Lightning Spear.

- (1) **Fireball:** The fireball must be thrown. On hit, it deals 20 damage and spawns a burning fire zone on the ground. If a character is standing in this zone, they are set on fire. This causes them to take 10 damage per second. Upon leaving the burning zone, the character continues to burn for an additional 3 seconds.
- (2) **Green Fireball:** This spell is essentially the same as the normal fireball, but green in color, deals more damage, and requires a different component for creation. It was primarily created to test the system in its early stages.
- (3) **Fire Arrow:** This is the most complex spell. It behaves like an arrow that must be drawn as if using a bow. While drawing, the fire grows and the visual representation changes from a fire bow into a large arrow. If the user releases it before it is sufficiently drawn, the fire simply disappears and the user must draw again. If it is drawn sufficiently, it is launched in the direction it is pointing.
- (4) **Lightning Spear:** The lightning spear must be thrown and deals 20 damage on hit.

2.3.14 Assets. To make the scene more immersive, different assets from the Unity Asset Store were modified and integrated into the scene. Overall, they have a low-poly style to ensure the efficiency of the program execution.

The landscape is based on the existing nature pack with low-poly style [2] that is portrayed on figure 12.

We implemented most spell animations, with the exception of the lightning spear, for which we used asset from the Unity Asset Store. Additionally, the 3D models used for spell components – a *coal*, a *feather*, a *battery* and a *flower* – were also integrated from the Unity Asset Store. The pictures of them are shown in Figure 13.

A primary challenge was the research and integration of these assets into the scene, as some of them used deprecated renderers or legacy materials that had to be manually changed to function correctly.

2.4 Connecting all Parts

2.4.1 Audio with Unity. To get the detected words over to Unity, we expanded our script to create a *TCP server* running on the local machine. This server would serve as the means of transport between the Python and Unity environment. We expanded the script to send every incoming word over this server.

For Unity to be able to receive these words, a client had to be constructed and the server started. We didn't want the player to have



Figure 11: Implemented spells in the demo



Figure 12: The scene landscape



Figure 13: The spell components

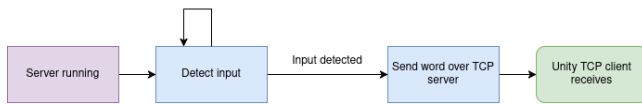


Figure 14: Diagram of running python script

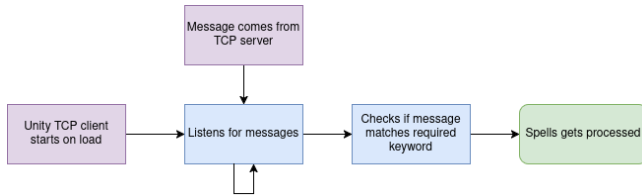


Figure 15: Diagram of the Unity client

to run a python script everytime they wanted to play, nonetheless have to install Python and dependencies, so we turned the script into a single executable. *PyInstaller* is responsible for bundling the code, the python executable and all libraries into a single package. This initially caused problems, as *vosk*'s model files were not being included inside the file. This was fixed by an adjustment to the spec file, adding in dynamic library collection. We then setup a Unity script to asynchronously read the servers input that stores the 3 last words in a queue, waiting to be processed further. These processes are demonstrated in figure 14 and 15.

Finally, we had to implement a GUI for changing the microphone used, as the player could not interact with the command line from the game world. Each change requires a restart of the server, as this was easier to implement and, as this process should usually only be done once per session, was a valid tradeoff.

2.4.2 Hardware Integration with Unity. To connect the glove to Unity, we used the original solution from the LucidVR Demo [6] as a basis. The problem is that it is implemented for the SteamVR SDK, whereas we are using OpenXR with the XR Interaction Toolkit. Therefore, the main function that applies force feedback had to be adapted. We also changed it conceptually.

Originally, the design was as follows: each interactable object was a client. In the SteamVR SDK, the grasping pose is baked into objects, so these objects, acting as clients, sent the hand grasping gesture for that object to the glove, which then adjusted the blockers (Section 2.1.1). However, since we decided to implement grasping the other way around, with the hand itself deciding whether it can take the object (Section 2.3.12), in our system the hand became the client.

Additionally, the hand subsystem from the XRI Toolkit already provides data about the curl of each finger in almost the correct format. The only processing we perform is converting the float values to integers and ensuring that force feedback is activated at the right moment for the correct fingers.

There is also an issue with the data from the potentiometers: they are slightly unstable. To address this, we introduced a small optimization that prevents the force feedback from blocking a finger if the curl value has not changed sufficiently.

3 Conclusion and results

We believe that we achieved the main goal of this project – the construction of a cost-effective haptic VR glove.

Despite the challenges regarding hardware assembly and its software integration, we believe that the project was a success. Even though the haptic feedback still requires some fine-tuning, the resulting embodiment seems to be higher than in standard VR controllers. To confirm this empirically, a user study needs to be conducted, for which we unfortunately did not have enough time.

One of main hurdles during the hardware assembly was the instability of the glove, which made the glove components detach frequently. The reason for that seems to be the softness of the glove material so in future a more rigid base glove could be used.

Another aspect of our project that could be improved on is the lack of content. We believe that with more time we could implement more features and spells that would make the scene even more immersive.

References

- [1] Kevin Carbotte. 2018. Noitom Enters Spatially Tracked Glove Market With Hi5 VR Glove Business Edition. <https://www.tomshardware.com/news/noitom-hi5-vr-gloves-available,36718.html>
- [2] JustCreate. 2023. Low-Poly Simple Nature Pack. <https://assetstore.unity.com/packages/3d/environments/landscapes/low-poly-simple-nature-pack-162153>
- [3] lucas vrtech. 2021. LucidGloves - Firmware and 3D Printer Files. <https://github.com/LucidVR/lucidgloves>
- [4] lucas vrtech. 2021. OpenGloves Driver. <https://github.com/LucidVR/opengloves-driver>
- [5] LucidVR. 2022. LucidGloves: VR Haptic Gloves on a budget. <https://hackaday.io/project/178243-lucidgloves-vr-haptic-gloves-on-a-budget>
- [6] LucidVR. 2026. OpenGloves Force Feedback Unity Demo. <https://github.com/LucidVR/opengloves-force-feedback-unity-demo> GitHub repository.
- [7] Paul Miles Preuschhoff, René Schäfer, Phillip Ahlers, Nele Preuß, David Gilbert, and Jan Borchers. 2026. TARDIS: Tabletop Augmented Reality for Dynamic Immersive Storytelling. In *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*. Association for Computing Machinery, New York, NY, USA, Article 634, 5 pages. doi:10.1145/3772363.3799299
- [8] Anastasios N Niarchos, Yannis Ioannidis, and Akrivi Katifori. 2023. The use of Immersive VR in Tabletop Role Playing Games as a Virtual Tabletop. (2023).
- [9] Raphael Palombo, Sebastian Weber, Marc Wyszynski, and Björn Niehaves. 2024. Glove versus controller: the effect of VR gloves and controllers on presence, embodiment, and cognitive absorption. *Frontiers in Virtual Reality* Volume 5 - 2024 (2024). doi:10.3389/frvir.2024.1337959
- [10] Phlegmati. 2026. SimpleGestureRecorder. <https://github.com/Phlegmati/SimpleGestureRecorder> GitHub repository.
- [11] Unity Asset Store. 2026. \$1 Point Cloud Gesture Recognizer. <https://assetstore.unity.com/packages/tools/input-management/pdollar-point-cloud-gesture-recognizer-21660> Unity Asset Store package.

Character Count

This work contains 25120 characters (without spaces).